LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF320803)
MINGGU 8 ACARA 20

SEMESTER 3



Tugas :
API Authentication dan Error Handling

Disusun Oleh:
Uaryl Engno Laurenca (E41250317)

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

A. PENDAHULUAN

Pada Acara 19 telah dibuat tabel `users` dan endpoint `POST /register`, sehingga sistem sudah bisa menyimpan akun pengguna. Namun seluruh endpoint API masih terbuka: siapa pun dapat menambah, mengubah, atau menghapus data mahasiswa tanpa identitas. Acara 20 menutup celah tersebut dengan autentikasi berbasis token.

Pengguna yang sudah terdaftar melakukan login dan menerima JSON Web Token (JWT). Token itu harus disertakan pada header `Authorization: Bearer <token>` setiap kali mengakses endpoint yang dilindungi. Karena RESTful API bersifat stateless, server tidak menyimpan sesi login, cukup memverifikasi tanda tangan (signature) token pada setiap request.

Acara ini juga membahas error handling: bagaimana mendeteksi, menangani, dan melaporkan kesalahan dalam format JSON yang konsisten tanpa membocorkan detail internal server kepada client.

B. TUJUAN PRAKTIKUM

1. Memahami struktur dan cara kerja JSON Web Token (header, payload, signature) sesuai RFC 7519.
2. Membuat class Auth pada lapisan Core yang menghasilkan dan memvalidasi token HS256 secara native (hash_hmac() dan Base64URL) tanpa pustaka eksternal.
3. Menambahkan aksi login() pada AuthController dan mendaftarkannya pada Router.
4. Melindungi endpoint POST, PUT, dan DELETE dengan middleware Auth::requireAuth(), sementara GET tetap publik.
5. Menerapkan try...catch pada lapisan Model dan memahami jenis kesalahan client (4xx) serta server (5xx).
6. Menguji alur autentikasi di Postman, termasuk kondisi gagal (tanpa token dan token diubah).
7. Menganalisis mengapa JWT stateless tidak memiliki logout di sisi server, dan mengamati token kedaluwarsa dengan masa berlaku 5 menit.

C. DASAR TEORI

3.1 Autentikasi pada RESTful API

Autentikasi adalah proses verifikasi identitas pengguna atau aplikasi sebelum diberi akses ke layanan. Tujuannya melindungi data dari akses tidak sah, memastikan identitas client, membatasi penggunaan endpoint tertentu, dan meningkatkan keamanan. REST API lebih sering memakai token karena sifatnya yang stateless. Mekanisme umum:

- API Key: kunci unik per client, dikirim lewat header, misalnya X-API-Key.
- Basic Authentication: username dan password di-encode Base64 pada header. Sederhana, tetapi sebaiknya hanya dipakai lewat HTTPS karena kredensial tetap bisa didekode.
- Bearer Token: token sebagai bukti client sudah login, dikirim lewat Authorization: Bearer <token>.

3.2 JSON Web Token (JWT)

JWT (RFC 7519) adalah token ringkas yang terdiri atas tiga bagian dipisah titik:

- Header: tipe token dan algoritma, misalnya {“alg”:”HS256",”typ”:”JWT”}.
- Payload: klaim (claims) data pengguna, misalnya id, role, waktu penerbitan (iat), dan waktu kedaluwarsa (exp).
- Signature: tanda tangan digital untuk memverifikasi keaslian token.

Karena stateless, server tidak perlu menyimpan sesi, cukup menghitung ulang signature pada setiap request. Payload hanya di-encode, tidak dienkripsi, sehingga dapat dibaca siapa pun. Signature menjamin keaslian, bukan kerahasiaan.

3.3 Implementasi Token Native pada PHP

Signature HS256 dapat dibuat tanpa Composer memakai hash_hmac('sha256', ..., true) yang dikombinasikan dengan base64_encode() yang disesuaikan menjadi Base64URL (karakter + dan / diganti - dan _, tanda = dibuang). Pendekatan ini sejalan dengan semangat PHP Native pada workshop, berbeda dari tutorial berbasis framework yang memakai pustaka seperti firebase/php-jwt.

3.4 Middleware Autentikasi

Middleware adalah lapisan perantara yang memeriksa suatu kondisi sebelum permintaan diteruskan ke logika utama. Pada PHP Native, middleware autentikasi berupa fungsi yang membaca header Authorization, memvalidasi token, lalu menghentikan proses dengan status 401 jika token tidak ada atau tidak valid.

3.5 Error Handling

Error handling adalah mekanisme mendeteksi, menangani, dan melaporkan kesalahan saat aplikasi berjalan. Kesalahan dibagi dua:

- Client error (4xx): kesalahan pada request, misalnya endpoint salah, parameter tidak lengkap, data tidak valid, atau tidak terautentikasi (400, 401, 403, 404).
- Server error (5xx): kesalahan di sisi server, misalnya koneksi database gagal atau konfigurasi tidak tepat (500).

PHP menyediakan blok try...catch untuk menangkap exception. Response kesalahan sebaiknya berformat JSON yang konsisten.

D. HASIL DAN PEMBAHASAN

4.1 Alat dan Bahan

XAMPP (Apache, PHP, MySQL), Postman, Visual Studio Code, Git dan GitHub, web browser.

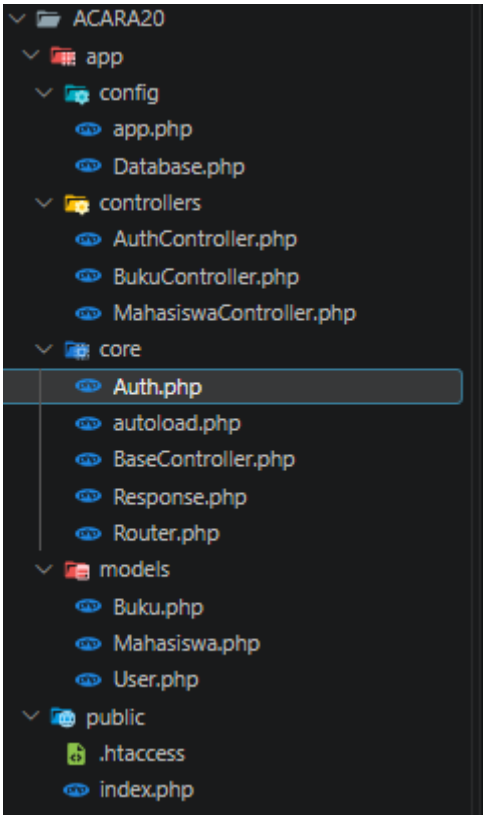
4.2 Gambaran Perubahan

Acara ini melengkapi fondasi Acara 19 dengan class Auth, aksi login, dan proteksi pada Controller.

- app/core/Auth.php : baru, pembuat dan pemeriksa token (JWT HS256) serta middleware requireAuth()
- app/controllers/AuthController.php : ditambah aksi login()

- app/core/Router.php : resource login dipetakan ke AuthController
- app/controllers/MahasiswaController.php : store, update, destroy memanggil Auth::requireAuth()
- app/controllers/BukuController.php : operasi tulis ikut dilindungi
- app/models/Mahasiswa.php : semua query dibungkus try...catch
- public/.htaccess : meneruskan header Authorization ke PHP
- Collection dan Environment Postman : diperbarui dengan folder Auth dan variabel token

Tabel database tidak berubah dari Acara 19.



[SCREENSHOT 1: struktur folder proyek di VS Code, terlihat Auth.php]

4.3 Langkah 1: Class Auth
Auth.php berisi tiga method utama.

a. generateToken(). Membuat header, payload (id, role, iat, exp), dan signature HMAC-SHA256 yang ditandatangani dengan kunci rahasia, lalu menggabungkannya menjadi header.payload.signature. Masa berlaku ditentukan konstanta TOKEN_TTL.

b. validateToken(). Memeriksa token melalui tiga tahap: (1) signature dihitung ulang dan dibandingkan dengan hash_equals() yang tahan timing attack; (2) algoritma pada header harus HS256; (3) payload harus memiliki exp yang belum lewat. Jika salah satu gagal, hasilnya false.

c. requireAuth(). Middleware yang membaca header Authorization, mengambil token dengan pola Bearer <token>, lalu memvalidasinya. Jika header tidak ada atau bukan Bearer, balasannya 401 “Token tidak ditemukan”. Jika token tidak sah atau

kedaluwarsa, balasannya 401 “Token tidak valid atau kedaluwarsa”. Kedua respons menyertakan header WWW-Authenticate: Bearer.
Beberapa perbaikan dibanding kode contoh modul: pembacaan payload memakai decode Base64URL yang benar (kode modul memakai base64_decode() biasa yang gagal bila payload menghasilkan karakter - atau _), pemeriksaan algoritma dan keberadaan exp, serta pembacaan header Authorization yang tidak peka huruf besar/kecil.

Kunci rahasia (SECRET) memakai nilai bawaan modul dan dapat ditimpa lewat environment variable JWT_SECRET. Kunci ini harus diganti sebelum deployment atau sebelum kode diunggah ke GitHub publik, karena siapa pun yang mengetahuinya dapat membuat token palsu.



[SCREENSHOT 2: kode Auth.php]

4.4 Langkah 2: Aksi login() pada AuthController
Urutan pemeriksaan pada login():

1. 400 Bad Request jika username atau password kosong, atau body bukan JSON valid.
2. Pengguna dicari dengan User::findByUsername(), lalu password dicocokkan dengan password_verify() terhadap hash di database.
3. 401 Unauthorized dengan pesan “Username atau password salah” jika pengguna tidak ada atau password salah. Pesannya sengaja sama agar penyerang tidak bisa mengetahui username mana yang terdaftar. password_verify() tetap dijalankan pada hash palsu saat pengguna tidak ditemukan, sehingga waktu respons tidak membedakan kedua kasus.
4. Jika berhasil, token dibuat dari id dan role pengguna, lalu dikirim dengan status 200 beserta token_type “Bearer”, expires_in 300, dan role.



```php
class AuthController extends BaseController {
    // Hash palsu: dipakai agar waktu respons login tetap mirip saat username tidak ada,
    // sehingga penyerang tidak bisa menebak username mana yang terdaftar dari lamanya respons.
    private const DUMMY_HASH = '$2y$10$410vGNYd7btt.0Zr1AmahOo04DF9pPXBhxoUp15vOu6kJ5w.FQToa';

    private User $model;

    public function __construct() {
        $this->model = new User();
    }

    public function register(): void {
        $in = $this->readJson();

        $data = [];
        foreach (['username', 'email', 'password'] as $field) {
            $value = $in[$field] ?? '';
            // password tidak di-trim: spasi di awal/akhir bisa jadi bagian dari password
            $data[$field] = is_scalar($value) ? ($field === 'password' ? (string) $value : trim((string) $value)) : '';
        }

        // 400: field wajib kosong / tidak dikirim
        $missing = [];
        foreach ($data as $field => $value) {
            if ($value === '') {
                $missing[$field] = "Field $field wajib diisi";
            }
        }
        if (!empty($missing)) {
            Response::json(400, false, 'Username, email, dan password wajib diisi', null, $missing);
        }

        // 422: format data tidak valid (termasuk email via filter_var di Model User)
        $errors = $this->model->validate($data);
        if (!empty($errors)) {
            Response::json(422, false, 'Data tidak lolos validasi', null, $errors);
        }

        // 409: username / email sudah terdaftar
        $conflict = [];
        if ($this->model->findByUsername($data['username'])) {
            $conflict['username'] = 'Username sudah dipakai';
        }
        if ($this->model->findByEmail($data['email'])) {
            $conflict['email'] = 'Email sudah terdaftar';
        }
        if (!empty($conflict)) {
            Response::json(409, false, 'Username atau email sudah terdaftar', null, $conflict);
        }

        // Password TIDAK PERNAH disimpan sebagai teks biasa
        $data['password'] = password_hash($data['password'], PASSWORD_DEFAULT);

        try {
            $id = $this->model->create($data);
        } catch (PDOException $e) {
            // Dua request bersamaan dengan data sama: constraint UNIQUE di database menolak yang kedua.
            if ($e->getCode() === '23000') {
                Response::json(409, false, 'Username atau email sudah terdaftar');
            }
            throw $e;
        }

        // Hash password tidak ikut dikembalikan ke client. Role selalu 'user' (default tabel).
        Response::json(201, true, 'Registrasi berhasil', [
            'id'       => (int) $id,
            'username' => $data['username'],
            'email'    => $data['email'],
            'role'     => 'user',
        ]);
    }

    public function login(): void {
        $in = $this->readJson();

        $username = isset($in['username']) && is_scalar($in['username']) ? trim((string) $in['username']) : '';
        $password = isset($in['password']) && is_scalar($in['password']) ? (string) $in['password'] : '';

        // 400: field tidak dikirim / kosong
        $missing = [];
        if ($username === '') {
            $missing['username'] = 'Field username wajib diisi';
        }
        if ($password === '') {
            $missing['password'] = 'Field password wajib diisi';
        }
        if (!empty($missing)) {
            Response::json(400, false, 'Username dan password wajib diisi', null, $missing);
        }

        $user = $this->model->findByUsername($username);

        // password_verify selalu dijalankan (pada hash palsu bila user tidak ada).
        $valid = password_verify($password, $user['password'] ?? self::DUMMY_HASH);

        // 401: pesan sengaja sama untuk "user tidak ada" dan "password salah"
        if (!$user || !$valid) {
            Response::json(401, false, 'Username atau password salah');
        }

        $token = Auth::generateToken(['id' => (int) $user['id'], 'role' => $user['role']]);

        Response::json(200, true, 'Login berhasil', [
            'token'      => $token,
            'token_type' => 'Bearer',
            'expires_in' => Auth::TOKEN_TTL,
            'role'       => $user['role'],
        ]);
    }
}
```

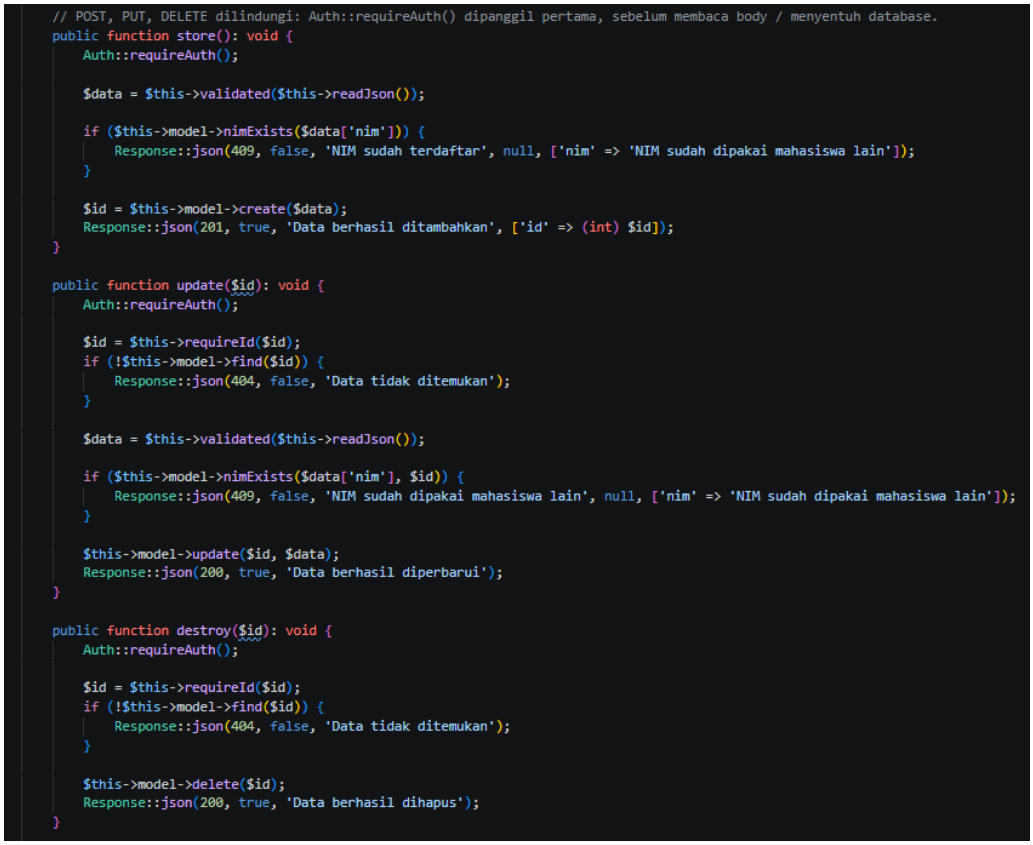
[SCREENSHOT 3: kode AuthController.php bagian login()]

4.5 Langkah 3: Mendaftarkan Resource login pada Router
Peta resource ditambah 'login' => 'AuthController'. Seperti register, login bukan resource CRUD: Router langsung memanggil aksinya dan menolak method selain POST dengan status 405 dan header Allow: POST.



[SCREENSHOT 4: kode Router.php]

4.6 Langkah 4: Melindungi Method Controller
Pada MahasiswaController, Auth::requireAuth() dipanggil pada baris pertama store(), update(), dan destroy(), sebelum body dibaca dan sebelum database disentuh. Method index() (GET) tetap publik. Perlindungan yang sama diterapkan pada operasi tulis BukuController agar konsisten.



```php
// POST, PUT, DELETE dilindungi: Auth::requireAuth() dipanggil pertama, sebelum membaca body / menyentuh database.
public function store(): void {
    Auth::requireAuth();

    $data = $this->validated($this->readJson());

    if ($this->model->nimExists($data['nim'])) {
        Response::json(409, false, 'NIM sudah terdaftar', null, ['nim' => 'NIM sudah dipakai mahasiswa lain']);
    }

    $id = $this->model->create($data);
    Response::json(201, true, 'Data berhasil ditambahkan', ['id' => (int) $id]);
}

public function update($id): void {
    Auth::requireAuth();

    $id = $this->requireId($id);
    if (!$this->model->find($id)) {
        Response::json(404, false, 'Data tidak ditemukan');
    }

    $data = $this->validated($this->readJson());

    if ($this->model->nimExists($data['nim'], $id)) {
        Response::json(409, false, 'NIM sudah dipakai mahasiswa lain', null, ['nim' => 'NIM sudah dipakai mahasiswa lain']);
    }

    $this->model->update($id, $data);
    Response::json(200, true, 'Data berhasil diperbarui');
}

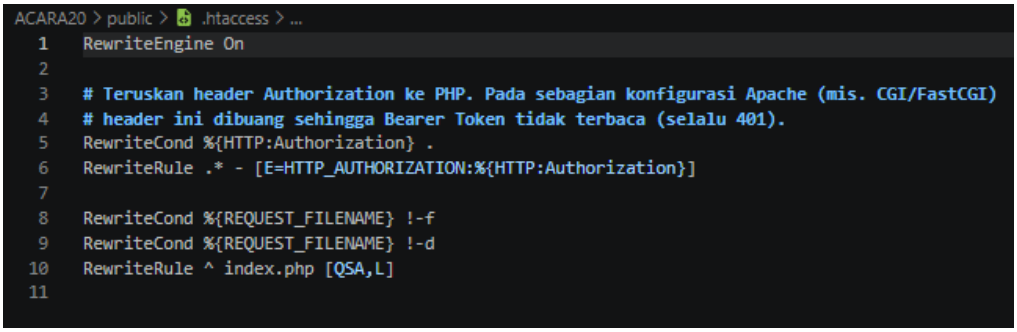
public function destroy($id): void {
    Auth::requireAuth();

    $id = $this->requireId($id);
    if (!$this->model->find($id)) {
        Response::json(404, false, 'Data tidak ditemukan');
    }

    $this->model->delete($id);
    Response::json(200, true, 'Data berhasil dihapus');
}
```

[SCREENSHOT 5: kode MahasiswaController.php, terlihat Auth::requireAuth() di store, update, dan destroy]

Pada public/.htaccess ditambahkan aturan yang meneruskan header Authorization ke PHP. Tanpa aturan ini, sebagian konfigurasi Apache membuang header tersebut sehingga token tidak terbaca dan request selalu mendapat 401.



ACARA20 > public > .htaccess > ...

```
1   RewriteEngine On
2
3   # Teruskan header Authorization ke PHP. Pada sebagian konfigurasi Apache (mis. CGI/FastCGI)
4   # header ini dibuang sehingga Bearer Token tidak terbaca (selalu 401).
5   RewriteCond %{HTTP:Authorization} .
6   RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]
7
8   RewriteCond %{REQUEST_FILENAME} !-f
9   RewriteCond %{REQUEST_FILENAME} !-d
10  RewriteRule ^ index.php [QSA,L]
11
```

[SCREENSHOT 6: isi public/.htaccess]

4.7 Langkah 5: Try-Catch pada Lapisan Model

Setiap query pada Mahasiswa.php dibungkus try...catch (PDOException). Saat terjadi kesalahan database, detail error (pesan SQL) dicatat ke log server melalui error_log(), sedangkan client hanya menerima status 500 dengan pesan umum “Terjadi kesalahan pada server”. Pelanggaran constraint UNIQUE (kode 23000), misalnya dua request dengan NIM sama masuk bersamaan, dibalas 409, bukan 500.

Ini berbeda dari contoh teori yang menampilkan $e->getMessage() langsung ke client. Pesan asli database dapat membocorkan nama tabel, struktur query, atau lokasi file, yang berguna bagi penyerang.

Penanganan berlapis yang dihasilkan:

- Model: menangkap PDOException, mencatat ke log, membalas 500 generik.
- Controller: validasi 400 dan 422, pemeriksaan 404 dan 409, serta 401 lewat requireAuth().
- Router: endpoint tidak dikenal (404) dan method salah (405).
- Front Controller (index.php): try...catch terakhir untuk error tak terduga, membalas 500 generik.
- Class Response: jika json_encode() gagal, tetap mengirim 500 berupa JSON yang sah.

ACARA20 > app > models > Mahasiswa.php > Mahasiswa

```php
<?php
/**
 * Model Mahasiswa. Acara 20: setiap query dibungkus try...catch (PDOException).
 * Detail error (pesan SQL) hanya masuk ke log server; client hanya menerima pesan umum 500.
 */
class Mahasiswa {
    private PDO $db;

    public function __construct() {
        $this->db = Database::getConnection();
    }

    /** Catat detail error ke log server, lalu balas 500 generik tanpa membocorkan SQL/nama tabel. */
    private function fail(PDOException $e): void {
        error_log('[Mahasiswa] ' . $e->getMessage());
        Response::json(500, false, 'Terjadi kesalahan pada server');
    }

    public function getAll(): array {
        try {
            return $this->db->query('SELECT * FROM mahasiswa ORDER BY id DESC')->fetchAll();
        } catch (PDOException $e) {
            $this->fail($e);
        }
    }

    public function find(int $id) {
        try {
            $stmt = $this->db->prepare('SELECT * FROM mahasiswa WHERE id = :id');
            $stmt->execute([':id' => $id]);
            return $stmt->fetch();
        } catch (PDOException $e) {
            $this->fail($e);
        }
    }

    /**
     * Cek apakah NIM sudah dipakai.
     * $exceptId dipakai saat UPDATE agar NIM milik data itu sendiri tidak dianggap duplikat.
     */
    public function nimExists(string $nim, ?int $exceptId = null): bool {
        try {
            if ($exceptId === null) {
                $stmt = $this->db->prepare('SELECT COUNT(*) FROM mahasiswa WHERE nim = :nim');
                $stmt->execute([':nim' => $nim]);
            } else {
                $stmt = $this->db->prepare('SELECT COUNT(*) FROM mahasiswa WHERE nim = :nim AND id <> :id');
                $stmt->execute([':nim' => $nim, ':id' => $exceptId]);
            }
            return (int) $stmt->fetchColumn() > 0;
        } catch (PDOException $e) {
            $this->fail($e);
        }
    }

    public function create(array $data): string {
        try {
            $stmt = $this->db->prepare(
                'INSERT INTO mahasiswa (nim, nama, prodi) VALUES (:nim, :nama, :prodi)'
            );
            $stmt->execute([':nim' => $data['nim'], ':nama' => $data['nama'], ':prodi' => $data['prodi']]);
            return $this->db->lastInsertId();
        } catch (PDOException $e) {
            // 23000 = pelanggaran UNIQUE (dua request dengan NIM sama masuk bersamaan)
            if ($e->getCode() === '23000') {
                Response::json(409, false, 'NIM sudah terdaftar', null, ['nim' => 'NIM sudah dipakai mahasiswa lain']);
            }
            $this->fail($e);
        }
    }

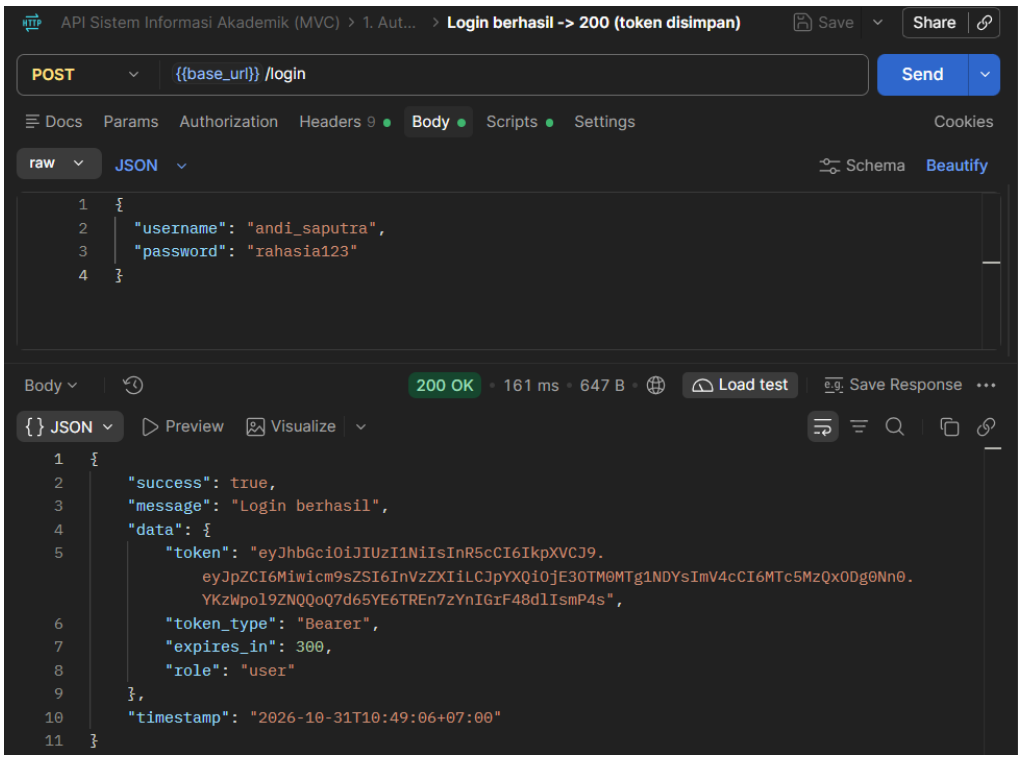
    public function update(int $id, array $data): void {
        try {
            $stmt = $this->db->prepare(
                'UPDATE mahasiswa SET nim = :nim, nama = :nama, prodi = :prodi WHERE id = :id'
            );
            $stmt->execute([':nim' => $data['nim'], ':nama' => $data['nama'], ':prodi' => $data['prodi'], ':id' => $id]);
        } catch (PDOException $e) {
            if ($e->getCode() === '23000') {
                Response::json(409, false, 'NIM sudah dipakai mahasiswa lain', null, ['nim' => 'NIM sudah dipakai mahasiswa lain']);
            }
            $this->fail($e);
        }
    }

    public function delete(int $id): void {
        try {
            $stmt = $this->db->prepare('DELETE FROM mahasiswa WHERE id = :id');
            $stmt->execute([':id' => $id]);
        } catch (PDOException $e) {
            $this->fail($e);
        }
    }
}
```

[SCREENSHOT 7: kode Mahasiswa.php, terlihat blok try...catch]

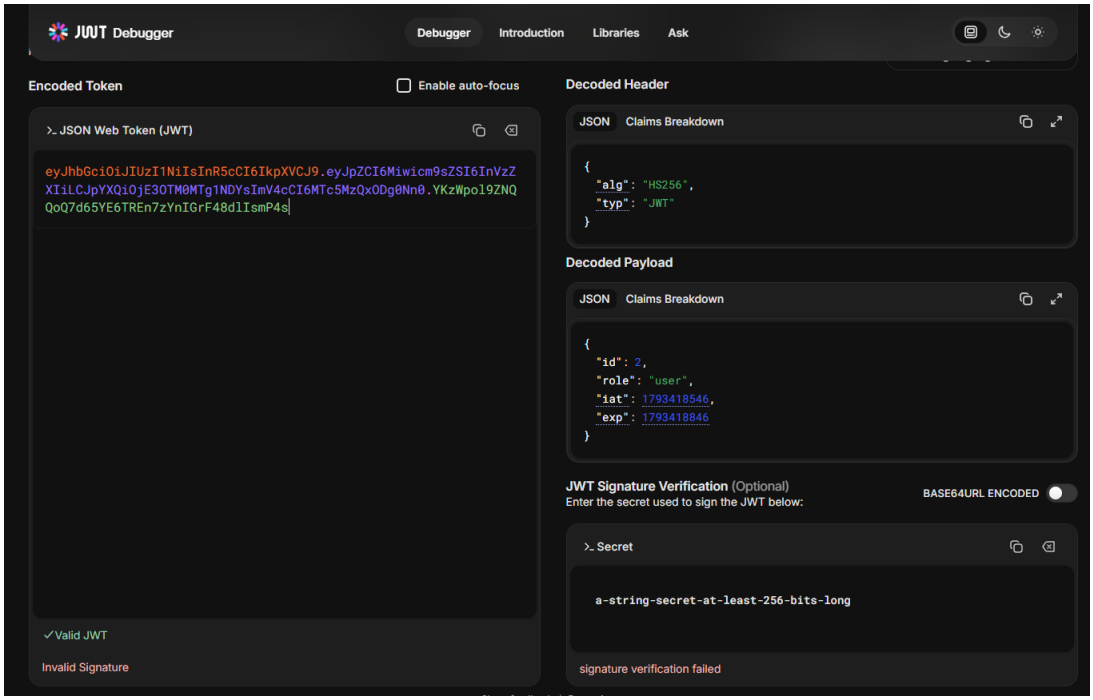
4.8 Langkah 6: Pengujian Alur Autentikasi di Postman
Base URL: http://localhost/ACARA16-25/ACARA20/public. Environment Local dipilih. Akun yang dipakai adalah hasil registrasi Acara 19 (username andi_saputra).

A. Login
(1) Login berhasil
POST {{base_url}}/login dengan
{“username”:”andi_saputra”,”password”:”rahasia123"}.
Hasil: status 200 OK, message “Login berhasil”, data berisi token (tiga bagian dipisah titik), token_type “Bearer”, expires_in 300, dan role “user”. Token disalin untuk pengujian berikutnya. Pada Collection, token disimpan otomatis ke variabel environment {{token}} oleh test script.



[SCREENSHOT 8: Postman login berhasil, status 200, terlihat token]

(2) Token diperiksa di jwt.io (opsional)
Bagian payload berisi id, role, iat, dan exp; selisih exp dan iat sama dengan 300 detik. Ini menunjukkan payload hanya di-encode, bukan dienkripsi.



[SCREENSHOT 9: token di jwt.io, terlihat header dan payload]

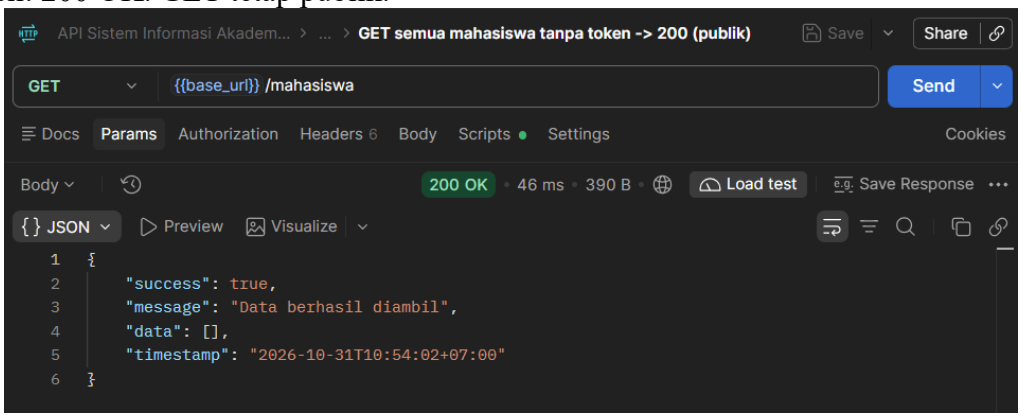
(3) Login gagal

- Password salah: status 401, "Username atau password salah".
  [SCREENSHOT 10: Postman login password salah, status 401]
- Username tidak terdaftar: status 401 dengan pesan yang sama.
- Field kosong: status 400 dengan errors per field.
- Method GET ke /login: status 405.

B. Mengakses endpoint dengan token

(1) GET /mahasiswa tanpa token

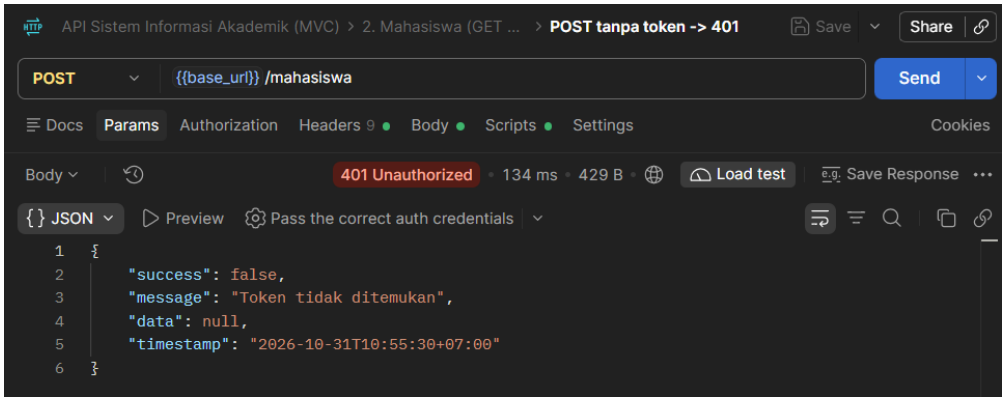
Hasil: 200 OK. GET tetap publik.



[SCREENSHOT 11: Postman GET /mahasiswa tanpa token, status 200]

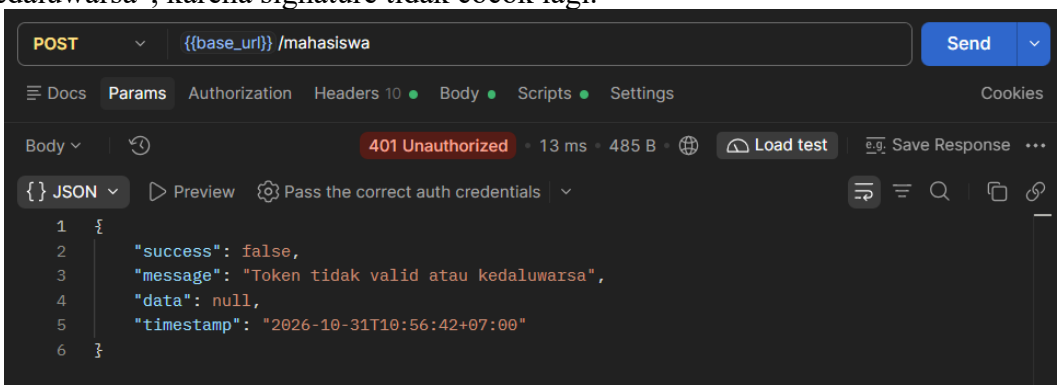
(2) POST /mahasiswa tanpa token

Hasil: 401 Unauthorized, message "Token tidak ditemukan".



[SCREENSHOT 12: Postman POST tanpa token, status 401]

(3) POST /mahasiswa dengan token yang sengaja diubah
Satu karakter pada token diubah atau ditambah. Hasil: 401, “Token tidak valid atau kedaluwarsa”, karena signature tidak cocok lagi.

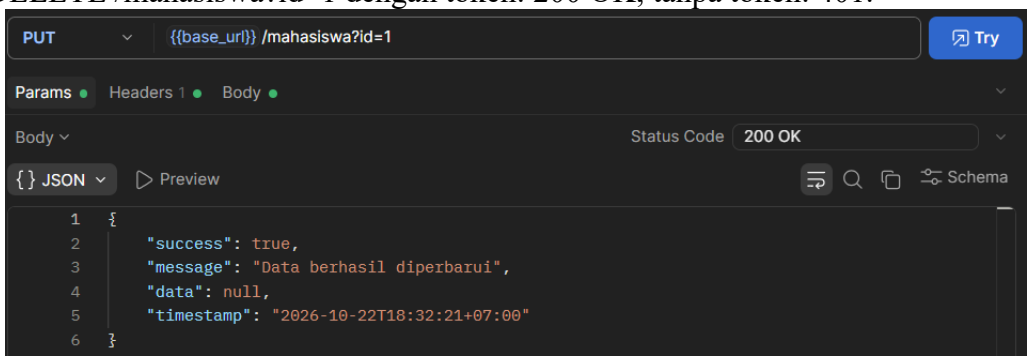


[SCREENSHOT 13: Postman POST dengan token diubah, status 401]

(4) POST /mahasiswa dengan token sah
Header Authorization: Bearer <token> (Postman: tab Authorization, Type Bearer Token). Body: {“nim”:”220100101",”nama”:”Andi Saputra”,”prodi”:”Teknik Informatika”}.
Hasil: 201 Created, data berisi id.

(5) PUT dan DELETE

- PUT /mahasiswa?id=1 dengan token: 200 OK; tanpa token: 401.
- DELETE /mahasiswa?id=1 dengan token: 200 OK; tanpa token: 401.

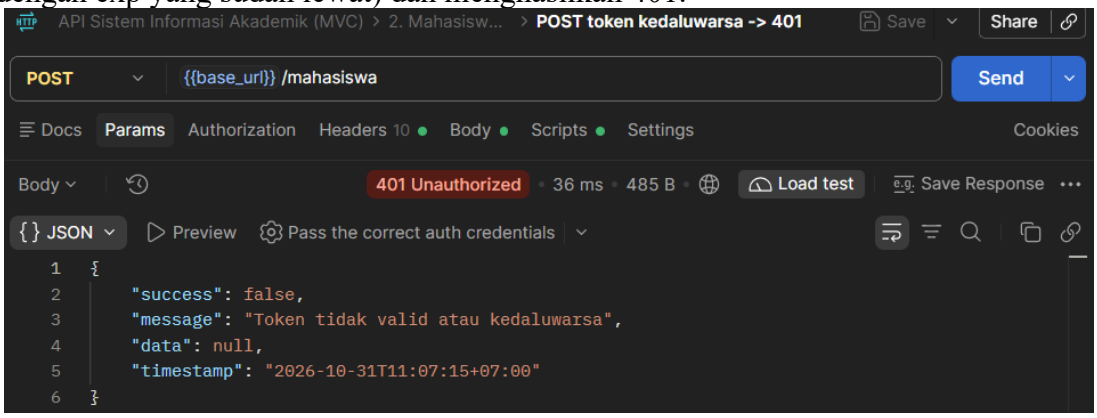




[SCREENSHOT 16: Postman PUT atau DELETE dengan token berhasil (200) dan tanpa token (401)]

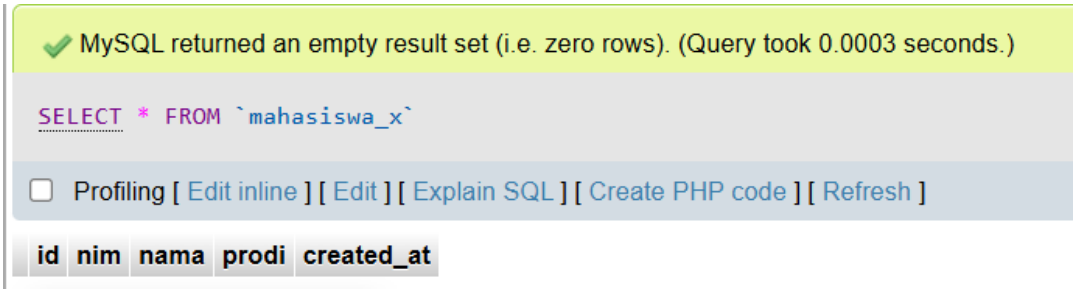
(6) Validasi tetap berjalan setelah lolos autentikasi
POST dengan NIM yang sudah ada menghasilkan 409, dan NIM bukan 9 karakter menghasilkan 422. Ini menunjukkan urutan pemeriksaan: autentikasi (401) lebih dulu, lalu validasi, lalu proses data.

C. Pengujian kedaluwarsa token (Tugas Mandiri 2)
Setelah login, ditunggu lebih dari 5 menit, lalu POST dikirim ulang dengan token yang sama. Hasil: 401 "Token tidak valid atau kedaluwarsa". Setelah login ulang, token baru berfungsi kembali (201). Sebagai alternatif tanpa menunggu, request "POST token kedaluwarsa" memakai variabel {{expired_token}} (token bertanda tangan benar dengan exp yang sudah lewat) dan menghasilkan 401.

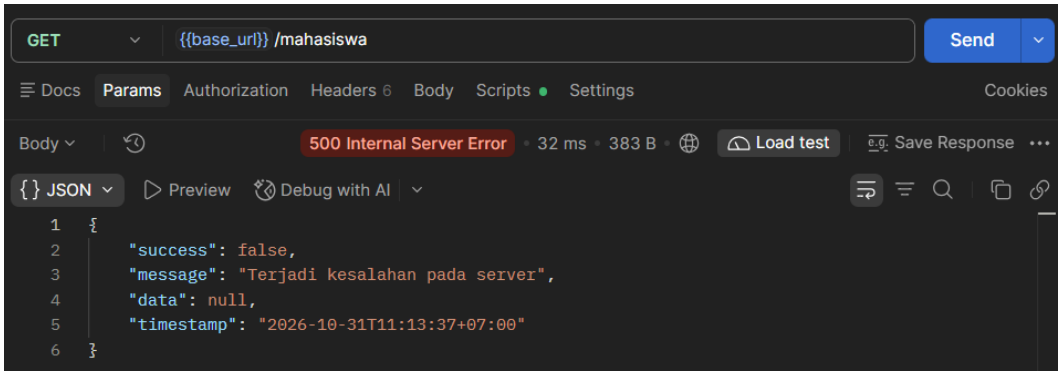


[SCREENSHOT 17: Postman POST dengan token kedaluwarsa, status 401]

D. Pengujian Error Handling Server
Nama tabel mahasiswa diubah sementara menjadi mahasiswa_x di phpMyAdmin. Request GET /mahasiswa dikirim. Hasil: 500 Internal Server Error dengan pesan umum "Terjadi kesalahan pada server" tanpa detail SQL. Detail "table doesn't exist" hanya tercatat di log error Apache (C:/laragon/tmp/php_errors.log). Nama tabel dikembalikan setelah pengujian.



[SCREENSHOT 18: phpMyAdmin, tabel diubah nama sementara]



[SCREENSHOT 19: Postman GET, status 500 dengan pesan umum]

E. Collection Runner

Seluruh request dijalankan berurutan (folder 1, 2, lalu 3). Setiap request memiliki test otomatis untuk status code, format response, header WWW-Authenticate pada respons 401, dan penyimpanan token.

4.9 Tugas Mandiri 1: Mengapa JWT Stateless Tidak Memiliki Logout di Sisi Server?

Penyebab. JWT bersifat self-contained: semua informasi untuk memeriksa token (identitas, role, waktu kedaluwarsa, dan tanda tangan) ada di dalam token itu sendiri. Server tidak menyimpan daftar sesi, sehingga setiap request cukup diverifikasi dengan menghitung ulang signature. Akibatnya, setelah token diterbitkan server tidak punya catatan tentang token itu yang bisa dihapus. Selama signature benar dan exp belum lewat, token dianggap sah, termasuk bila pengguna sudah menekan "logout" dan bila token dicuri.

Yang terjadi saat "logout". Logout hanya terjadi di sisi client, yaitu aplikasi membuang token dari penyimpanannya. Token yang sama, bila masih tersimpan di tempat lain atau sudah disalin pihak lain, tetap berlaku sampai exp.

Mekanisme tambahan bila server harus bisa membatalkan token:

- Masa berlaku pendek (praktikum ini 5 menit): jendela penyalahgunaan kecil, tetapi pengguna harus login ulang lebih sering.
- Access token pendek ditambah refresh token yang disimpan di database dan bisa dicabut: logout berarti mencabut refresh token, sementara access token tetap hidup sampai exp.
- Blacklist token: server menyimpan ID token (jti) yang dibatalkan sampai exp-nya lewat, dan requireAuth() memeriksanya. Konsekuensinya server kembali menyimpan state dan ada satu query tambahan per request.

- Versi token per pengguna: kolom token_version di tabel users dimasukkan ke payload, dan logout menaikkan angkanya sehingga semua token lama langsung tidak sah. Konsekuensinya butuh pembacaan database tiap request.

Kesimpulan. Tidak adanya logout di sisi server adalah konsekuensi langsung dari desain stateless, bukan kekurangan implementasi. Kemudahan skala (tanpa penyimpanan sesi) dibayar dengan ketidakmampuan mencabut token sebelum waktunya. Pada praktikum ini risikonya dikendalikan dengan masa berlaku 5 menit. Untuk sistem nyata yang membutuhkan logout atau pencabutan akses seketika, dipakai kombinasi token pendek dengan refresh token atau blacklist, dengan menerima bahwa sebagian sifat stateless hilang.

4.10 Tugas Mandiri 2: Masa Berlaku Token 5 Menit
Konstanta TOKEN_TTL pada Auth.php diubah dari 3600 detik (1 jam) menjadi 300 detik (5 menit) dan dipakai oleh generateToken(). Pengamatannya ada pada bagian 4.8 C. Respons login juga menampilkan expires_in 300 agar client mengetahui masa berlaku token.

4.11 Pembahasan

1. Autentikasi dilakukan sebelum logika lain. Auth::requireAuth() dipanggil di baris pertama method yang dilindungi, sehingga request tanpa token ditolak sebelum body dibaca dan sebelum database diakses.
2. Signature menjamin keaslian. Mengubah payload atau signature membuat hash yang dihitung ulang server tidak cocok, sehingga token ditolak. Perbandingan memakai hash_equals() agar tahan timing attack. Namun payload tetap dapat dibaca siapa pun, jadi data rahasia tidak boleh ditaruh di dalamnya.
3. Pemeriksaan algoritma. Header dengan algoritma selain HS256 ditolak, untuk mencegah serangan yang memanipulasi header agar melewati verifikasi.
4. Pesan login yang seragam. Pesan 401 yang sama untuk username tidak ada dan password salah menghindari kebocoran informasi tentang akun yang terdaftar.
5. Perbedaan 401 dan 403. 401 dipakai ketika identitas tidak terbukti (tanpa token atau token tidak sah), sedangkan 403 dipakai ketika identitas sah tetapi tidak berhak. Kode 403 belum dipakai karena belum ada pembatasan berdasarkan role, tetapi role sudah ada di payload sehingga dapat dipakai pada pengembangan berikutnya.
6. Pesan error untuk client tidak boleh membocorkan detail internal. Detail error ke log, pesan umum ke client. Ini dibuktikan pada pengujian 4.8 D.
7. GET dibiarkan publik sesuai modul untuk kebutuhan pengujian acara berikutnya. Pada sistem nyata, data yang sensitif sebaiknya juga dilindungi.
8. Kunci rahasia adalah titik lemah utama. Siapa pun yang mengetahui SECRET dapat membuat token palsu, jadi kunci harus diganti dari nilai contoh, disimpan di luar kode sumber (environment variable), dan tidak diunggah ke repositori publik.
9. Token dikirim lewat header, bukan URL, agar tidak tercatat di log akses dan riwayat browser. Pada penggunaan nyata, token harus dikirim lewat HTTPS.

E. KESIMPULAN

Class `Auth` berhasil dibuat menggunakan JWT HS256 native dengan fungsi `hash_hmac()` dan Base64URL tanpa pustaka eksternal, dilengkapi middleware

`requireAuth()` untuk memvalidasi token. Aksi `login()` berhasil menghasilkan token setelah kata sandi diverifikasi menggunakan `password_verify()`, dan resource login telah terdaftar pada Router. Endpoint POST, PUT, dan DELETE berhasil dilindungi sehingga permintaan tanpa token, dengan token yang telah dimodifikasi, atau dengan token kedaluwarsa akan mendapatkan respons HTTP 401, sedangkan endpoint GET tetap dapat diakses secara publik. Penanganan error berlapis pada Model, Controller, Router, Front Controller, dan Response juga berhasil diterapkan untuk menghasilkan pesan JSON yang konsisten, dengan detail kesalahan disimpan dalam log server dan client hanya menerima pesan umum. Tugas mandiri turut terpenuhi dengan memahami bahwa JWT stateless tidak mendukung logout server secara langsung karena server tidak menyimpan status token; pencabutan token memerlukan mekanisme tambahan seperti blacklist. Selain itu, masa berlaku token selama 5 menit berhasil diverifikasi melalui respons HTTP 401 ketika token telah kedaluwarsa.

.